



Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Clustering

Finding the groups nobody labelled: K-means, the cost it minimises, and how many clusters to ask for

Clustering

When dealing with complex datasets of many observations and variables, clustering helps us understand the underlying structure by grouping data points with similar characteristics into distinct clusters.

Customer Segmentation

Grouping customers by purchasing behaviour, preferences or demographics, for targeted marketing strategies.

Speech and Audio

Clustering audio data for speaker identification, music genre classification and speech analysis.

Image Segmentation

Dividing images into meaningful segments based on similarity, which supports object recognition and computer vision tasks.

E-commerce

Grouping products or users by purchasing patterns, to optimise inventory and improve the user experience.

Text Document Clustering

Grouping similar documents for topic modelling, document organisation and information retrieval.



The Old Faithful Dataset

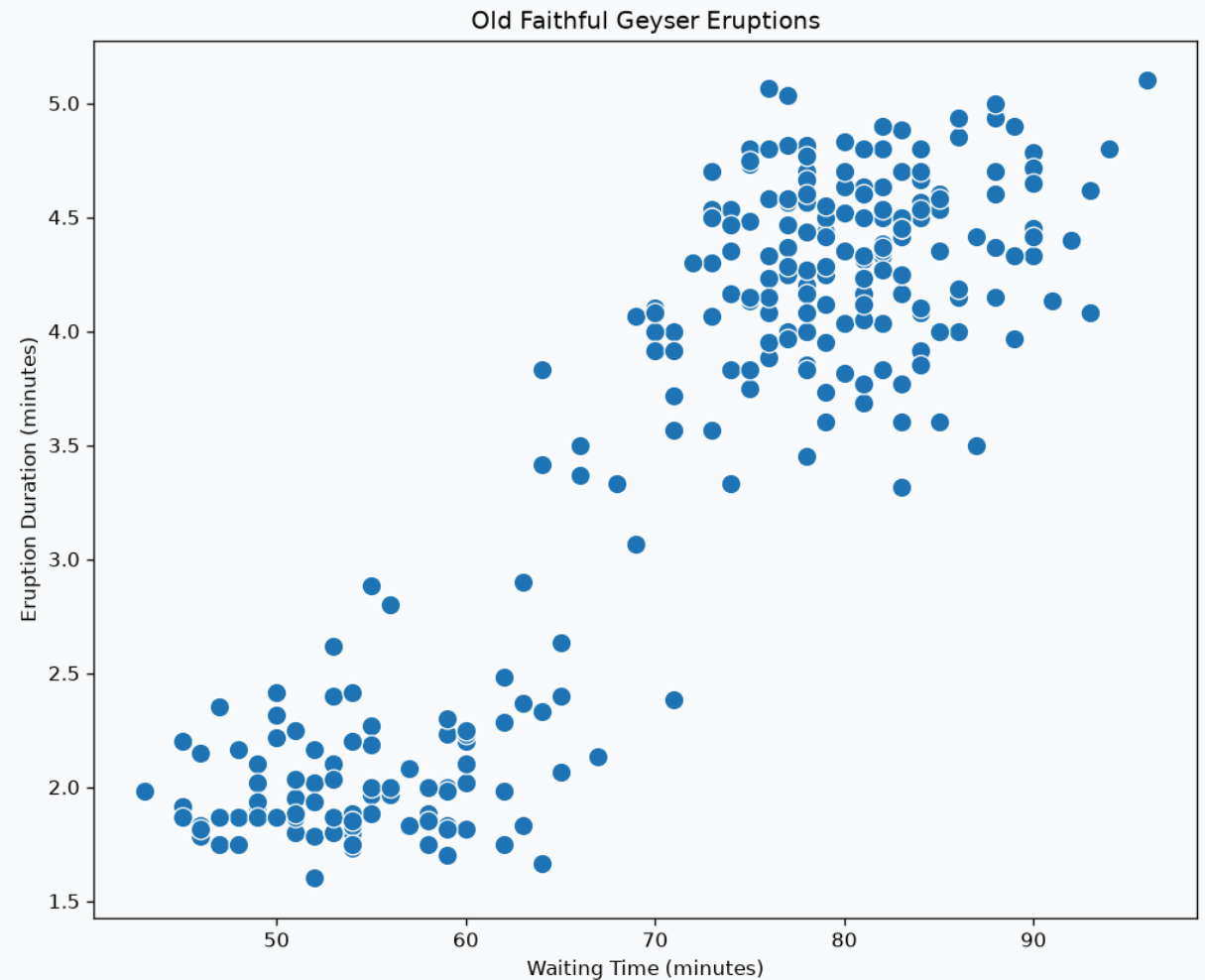
The Old Faithful geyser at Yellowstone National Park gives a clean example of natural clustering. The dataset holds **272 observations**, each with two variables:

- the **duration** of the eruption, in minutes;
- the **time to the next eruption**, in minutes.

Plotted as 2-D vectors, two groups are plain to see:

- short eruptions followed by a short wait (bottom left);
- long eruptions followed by a long wait (upper right).

A clustering algorithm has to find these two groups on its own. Note that it will call them "cluster 1" and "cluster 2": the algorithm assigns no meaning to a cluster, it only puts similar points together.



Formal Problem Definition

Let

$$\mathbf{X} = \left\{ \mathbf{x}^{(i)} \right\}_{i=1}^N, \mathbf{x}^{(i)} \in \mathcal{R}^n$$

be a set of N observations, each an n -dimensional vector. The goal of clustering is to split $\mathbf{X}$ into K groups, the clusters

$$\mathcal{S} = \{S_1, S_2, \dots, S_K\}$$

such that:

$$\forall \mathbf{x}^{(i)} \in \mathbf{X} \exists! S_j \in \mathcal{S} : \mathbf{x}^{(i)} \in S_j$$

Every data point belongs to **exactly one** cluster, elements within a group are similar to each other, and elements of different groups are dissimilar.

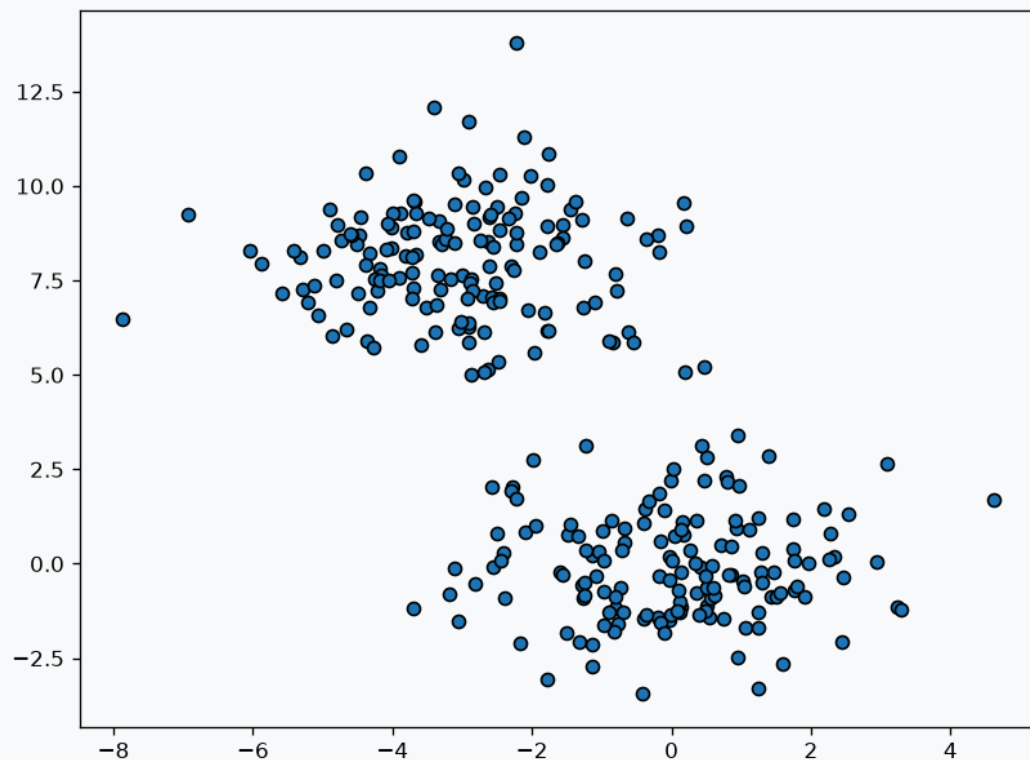
- **Key insight:** the number of clusters K is usually a **hyper-parameter**, which has to be specified before the algorithm runs.



From Unlabeled to Labeled Data

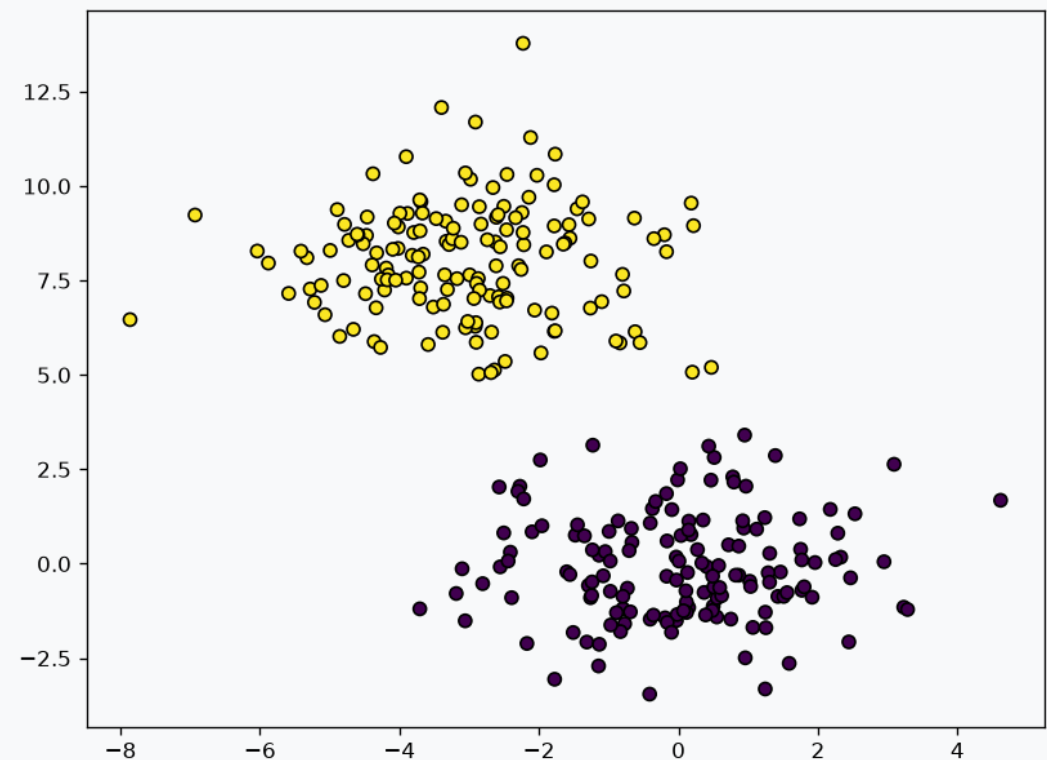
A graphical example

Before clustering



Data points exist without group assignments: no discrete variable tells us which group each point belongs to.

After clustering



The algorithm has identified two groups, so that points within each group are similar and points from different groups are not.

Other partitions of these points are possible. The one drawn here is the one that keeps the Euclidean distance small *within* each group and large *between* groups — which is exactly what the next algorithm will optimise.



K-Means Clustering

K-means attempts to create K clusters which are **as compact as possible**, where K is specified by the user.

We start by defining K vectors $\mu_k \in \mathbb{R}^n$, which act as prototypes for our clusters — we will see shortly that they are the clusters' centres. We then define $N \times K$ binary variables r_{ij} , mapping data points $\mathbf{x}^{(i)}$ to clusters S_j :

$$r_{ij} = \begin{cases} 1 & \text{if } \mathbf{x}^{(i)} \in S_j \\ 0 & \text{otherwise} \end{cases}$$

The notion that clusters should be as compact as possible is formalised by a **cost function**, sometimes called the **distortion function**:

$$J = \sum_{j=1}^K \sum_{i=1}^N r_{ij} \|\mathbf{x}_i - \mu_j\|^2$$

This J is the total **Within-Cluster Sum of Squares (WCSS)**. For a single cluster S_j , the term $\sum_{i=1}^N r_{ij} \|\mathbf{x}_i - \mu_j\|^2$ is the sum of squared distances of its points from its centre — a measure of the cluster's **inertia**, or compactness.

The goal is to find the centroids $\{\mu_j\}$ and the assignments $\{r_{ij}\}$ that minimise the total inertia:

$$\{\hat{r}_{ij}\}_{ij}, \{\hat{\mu}_j\}_j = \arg \min \sum_{j=1}^K \sum_{i=1}^N r_{ij} \left\| \mathbf{x}^{(i)} - \mu_j \right\|^2$$



The K-Means Algorithm

Optimization

The minimisation above has no closed form, so it is solved iteratively: start from K random centroids, then alternate two steps that can each only lower the cost.

1

Initialize

Randomly choose K cluster centroids $\mu_1, \mu_2, \dots, \mu_K$.

2

Assignment Step

Assign each point to the nearest centroid, by Euclidean distance.

3

Update Step

Recompute each centroid as the mean of the points assigned to it.

4

Repeat

Iterate until convergence, or until a maximum number of iterations.

The algorithm converges when the update step moves no centroid. In some cases it may never quite converge, which is why a maximum number of iterations is a usual termination criterion.

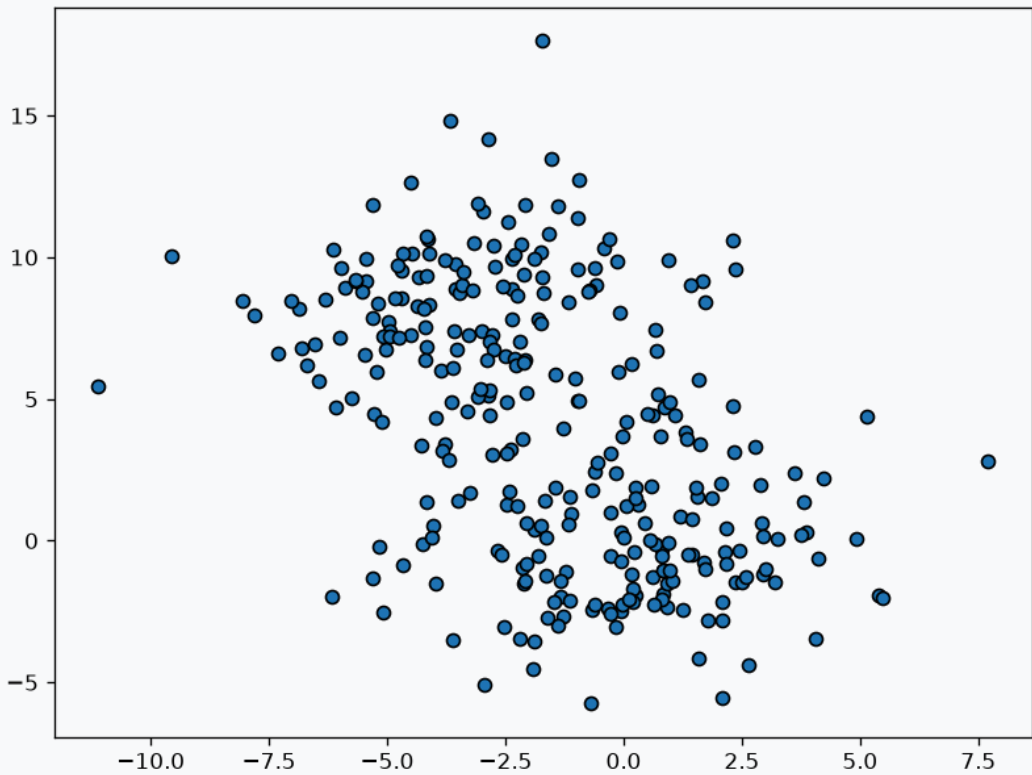
- **Important:** the algorithm is not guaranteed to find the global optimum — the solution may depend on the random initialization. In practice it usually leads to a reasonable result.



K-Means in Action: Initialization

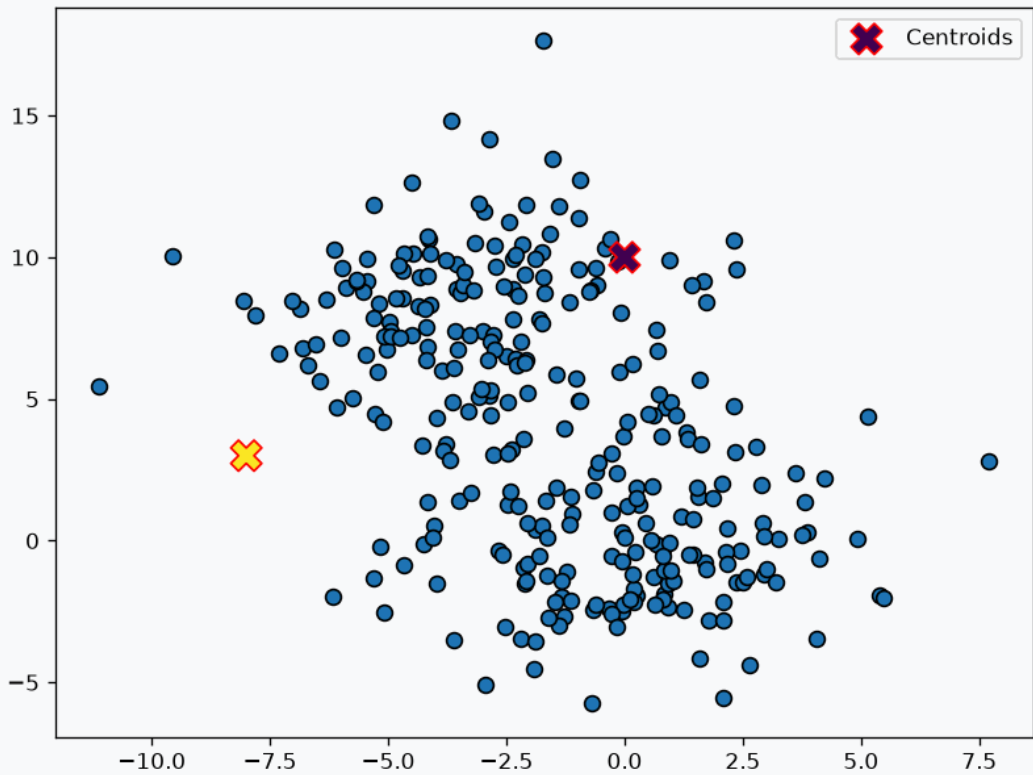
Example execution, step by step

The dataset



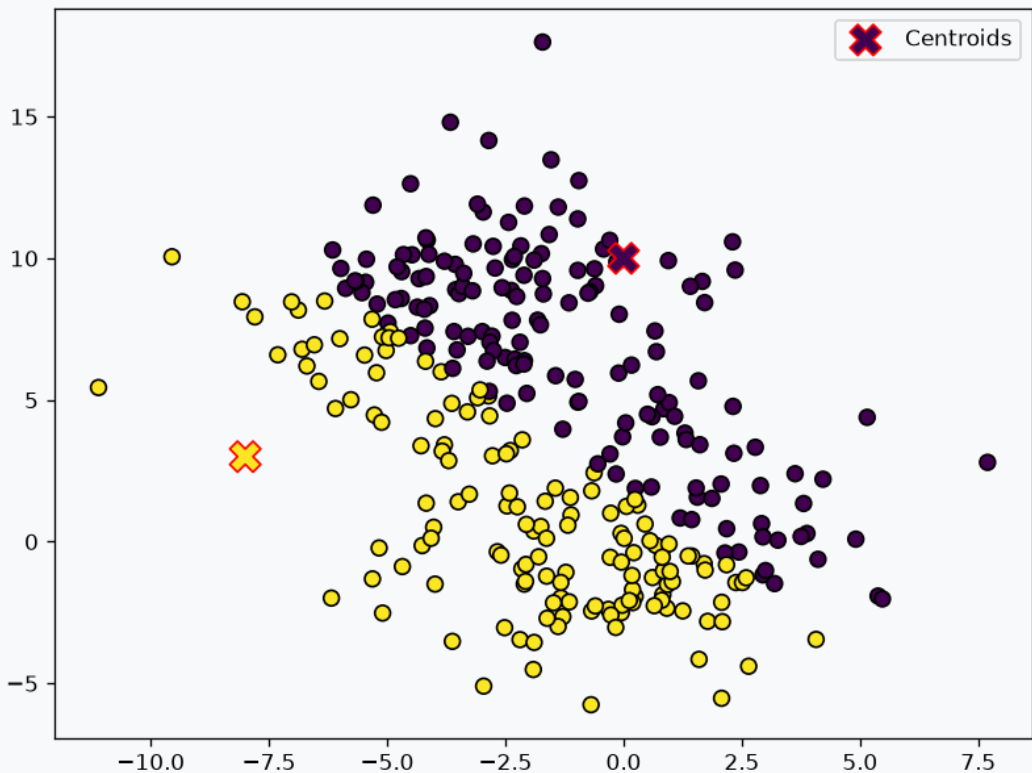
300 points drawn around two centres, with no labels.

Random initialization



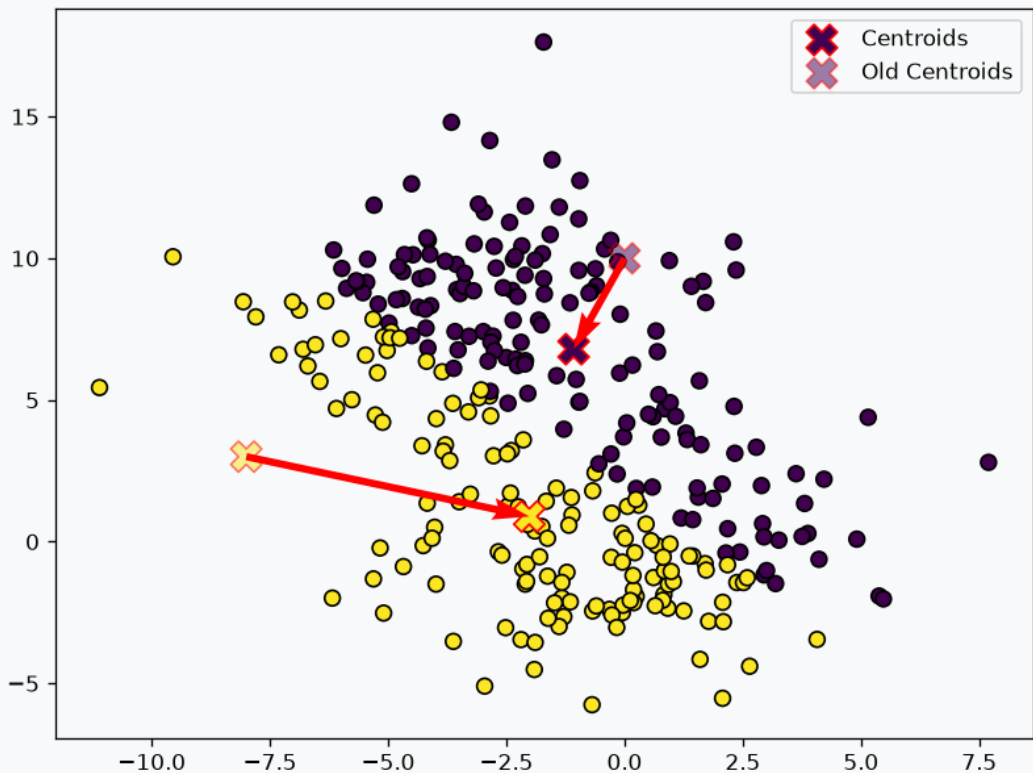
Two centroids are placed at random. Neither of them is anywhere near the middle of a group.

Assignment (1)



Every point takes the colour of its nearest centroid. The partition this produces is bad, because the centroids are badly placed.

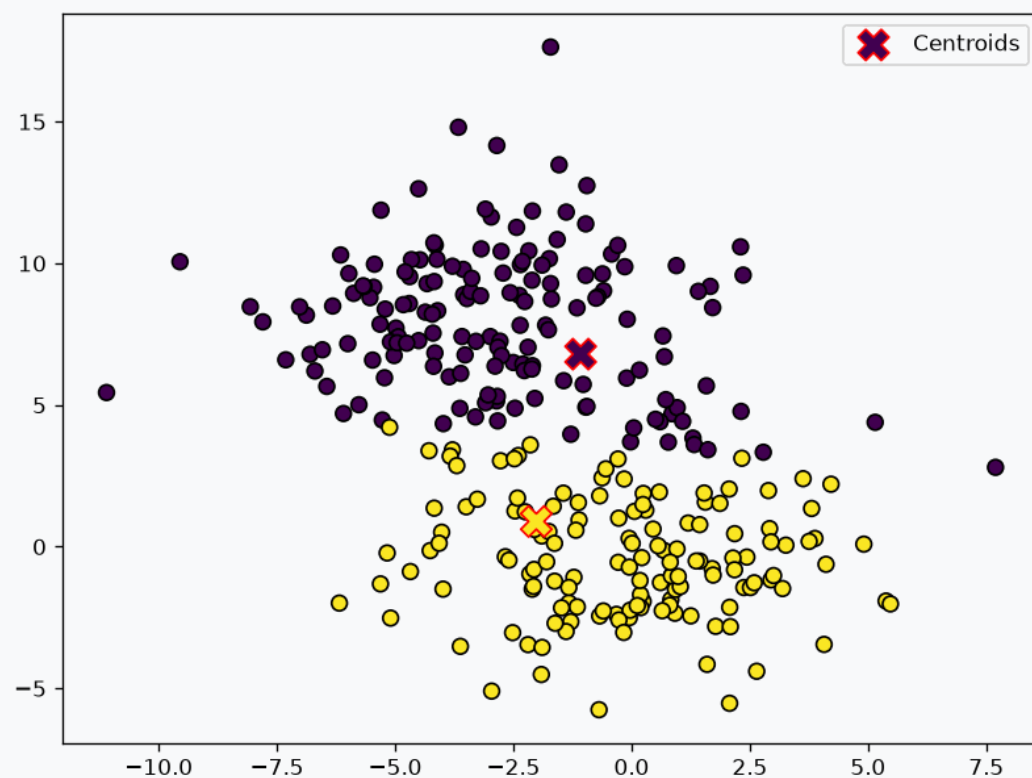
Update (1)



Each centroid moves to the mean of its own points — the red arrows. Both jump a long way at this first step.

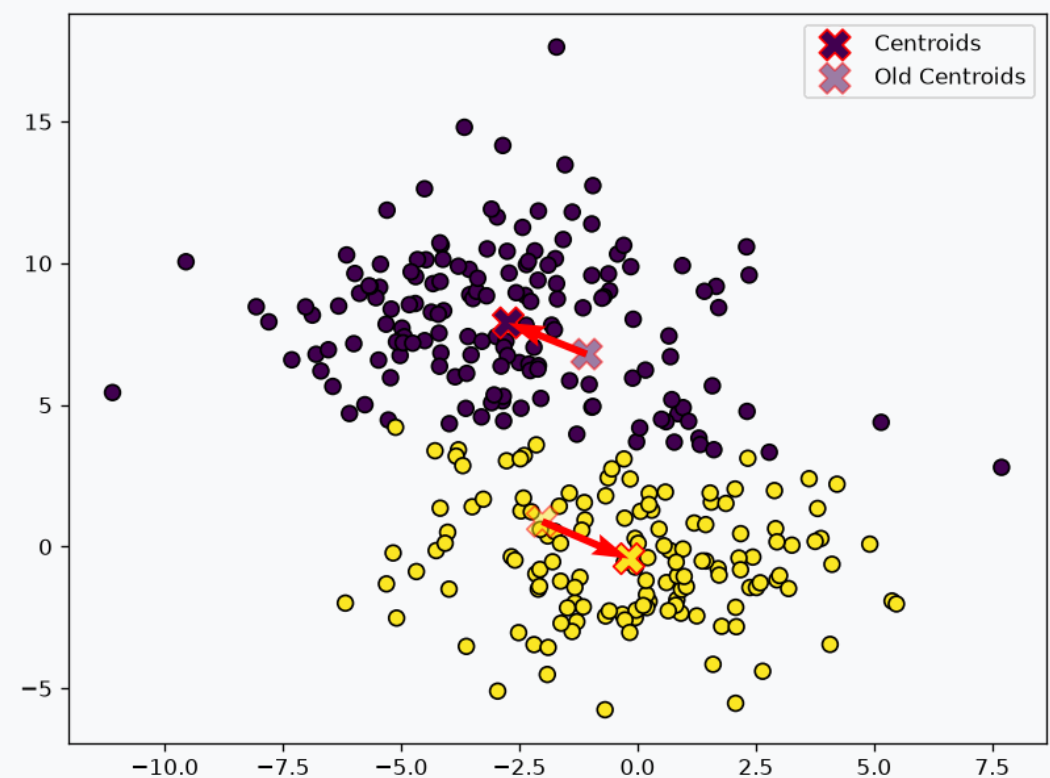
K-Means in Action: Iterating

Assignment (2)



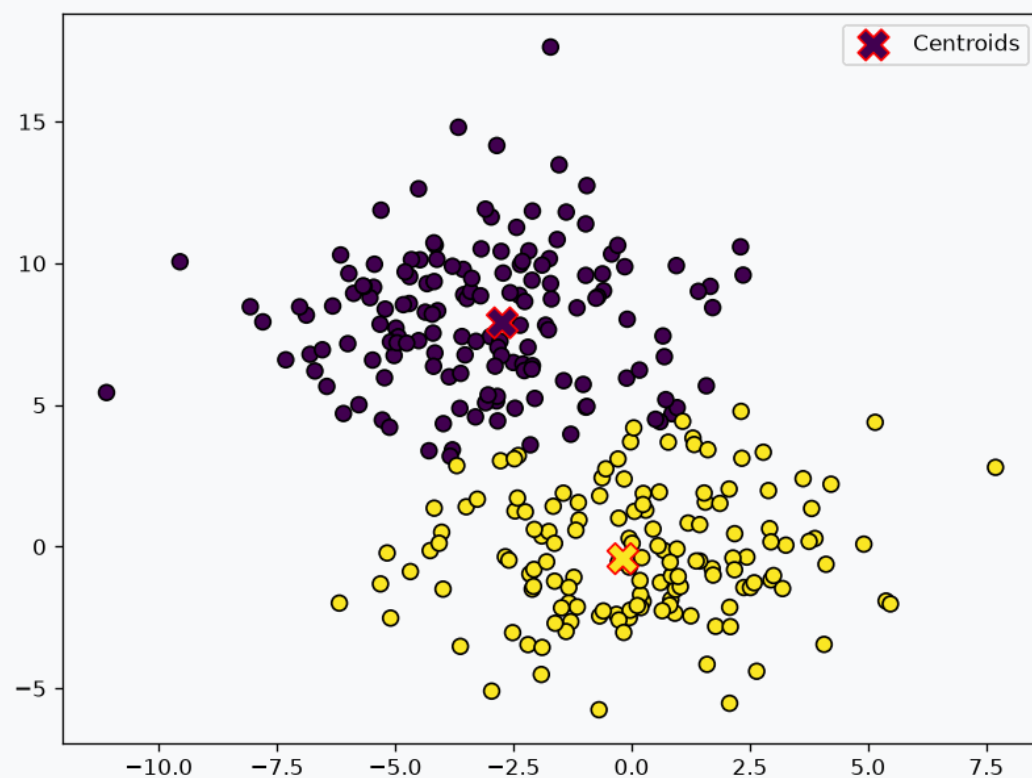
With the centroids in better positions, the boundary between the two colours is already close to the one we would draw by eye.

Update (2)



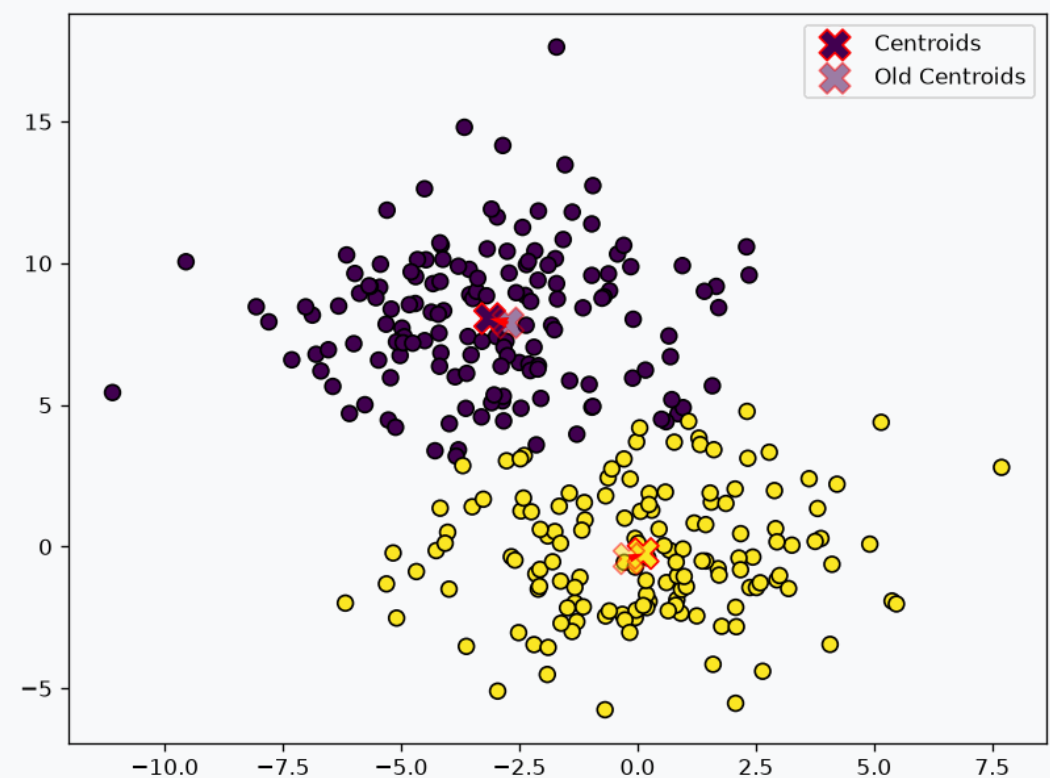
The centroids move again, by much less than before.

Assignment (3)



Only a few points on the border change colour.

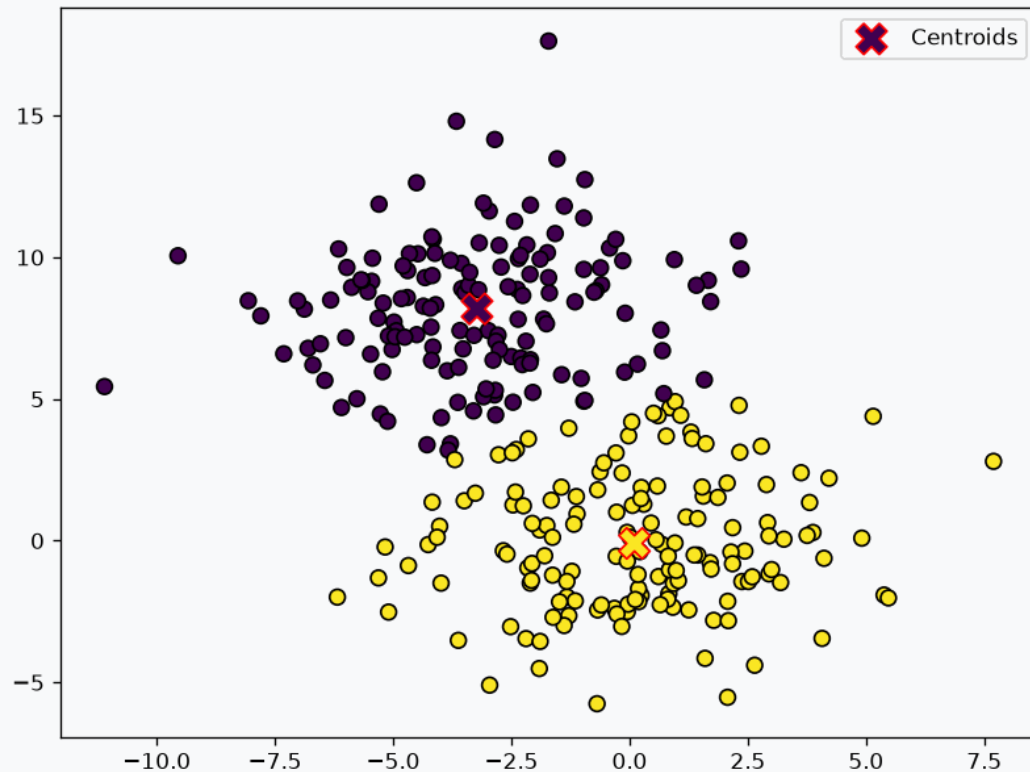
Update (3)



The arrows are now so short that the old and the new centroid overlap.

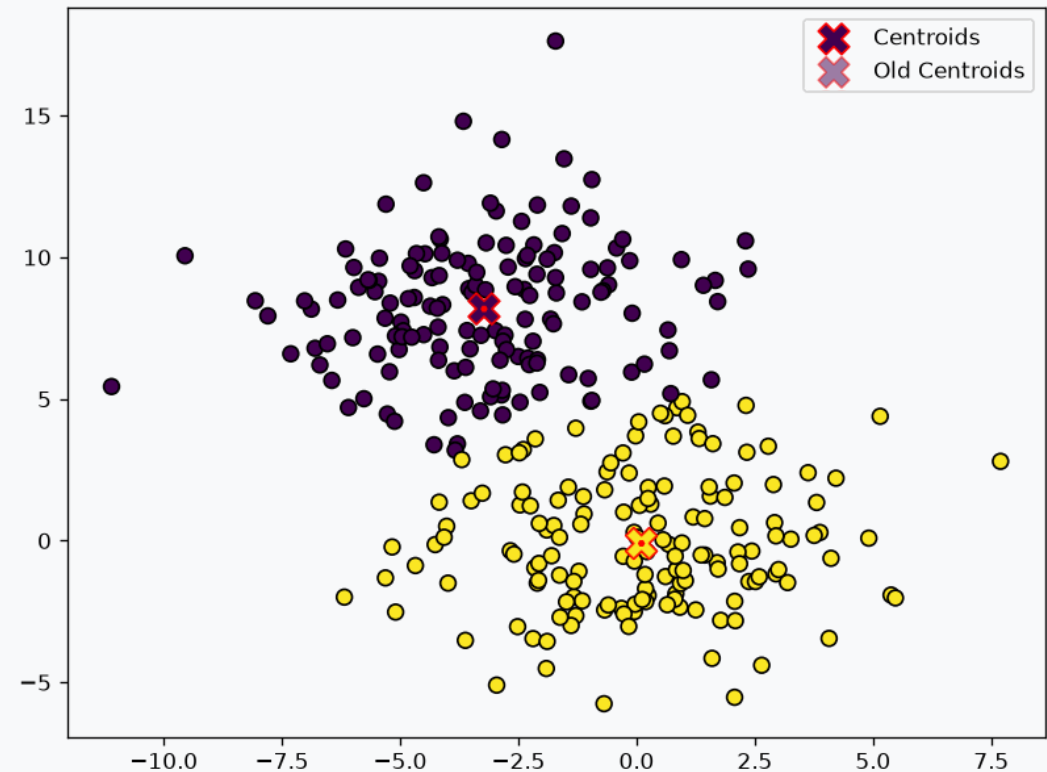
K-Means in Action: Convergence

Assignment (4)



No point changes cluster.

Update (4)

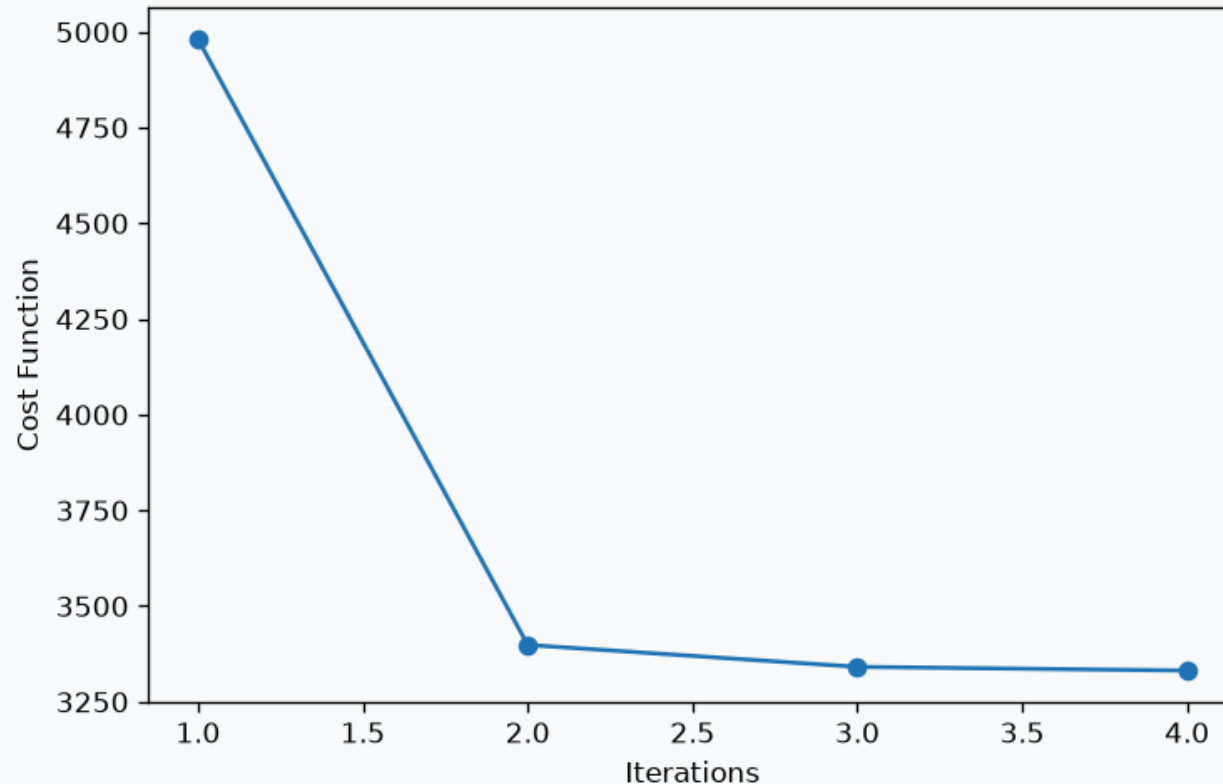


No centroid moves. The optimisation terminates: the data is clustered into two groups.

Each pair of steps can only lower the cost J : the assignment step gives every point the nearest centroid it could have, and the update step gives every cluster the centre that minimises its inertia. That is why the algorithm stops, even without any guarantee that where it stops is the global optimum.



Convergence and Cost Function



The cost function at the end of each iteration. The first update does most of the work — from about **4 980 to 3 400** — and the last two iterations barely change it at all.

After iteration 4 the centroids no longer move during the update step: the algorithm has converged, and the data is clustered into two groups with minimal within-cluster variance.



Pseudocode

Randomly initialize K cluster centroids $\mu_1, \mu_2, \dots, \mu_K \in \mathcal{R}^n$

Repeat until the termination criterion is reached {

for $i = 1$ to N — **assignment**

$$r_{ij} = \begin{cases} 1 & \text{if } j = \arg_j \min \|\mathbf{x}^{(i)} - \mu_j\|^2, \forall i, j \\ 0 & \text{otherwise} \end{cases}$$

for $j = 1$ to K — **update**

$$\mu_j = \frac{\sum_{i=1}^N r_{ij} \mathbf{x}^{(i)}}{\sum_{i=1}^N r_{ij}}$$

}

The assignment step produces the partition

$$S_j = \left\{ \mathbf{x} \in \mathbf{X} : \|\mathbf{x} - \mu_j\|^2 \leq \|\mathbf{x} - \mu_l\|^2 \quad \forall l \in \{1, \dots, K\} \right\}$$

and the update step is just the mean of each set, $\mu_j = \frac{1}{|S_j|} \sum_{\mathbf{x} \in S_j} \mathbf{x}$.



Choosing the Right K

The number of clusters K is a **hyper-parameter**: it affects the model we obtain, and the optimisation does not determine it for us. The usual approach is to fit several models with different guesses and keep the best one. Two techniques help:

Elbow Method

Plot the cost function for different values of K and look for the "**elbow point**", where the rate of decrease slows sharply. Past it, more clusters buy little.

Silhouette Method

Measure **how well each point fits its assigned cluster** compared with the others. Higher average silhouette scores mean better-defined, better-separated clusters.

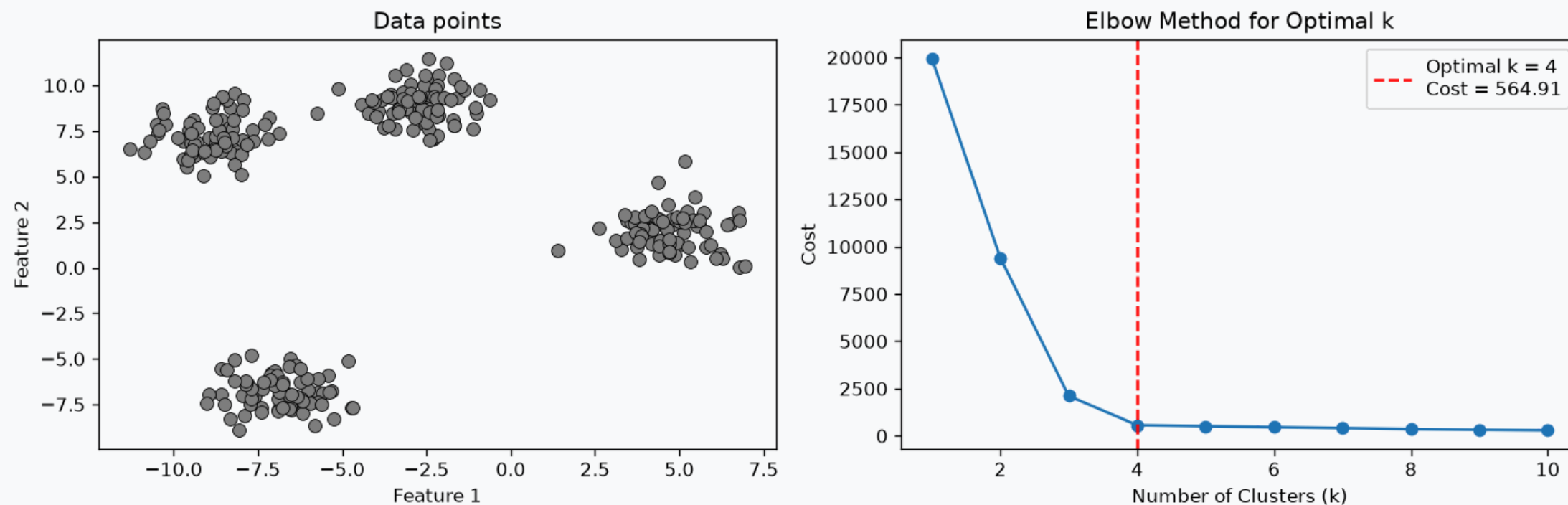
- Both are **heuristics**, and neither guarantees the optimal K . They are guidance, most useful when combined with an intuition of what a good K would look like for the problem at hand.



The Elbow Method Explained

The elbow method fits K-means for different values of K and plots the final value of the cost function. As K grows the cost falls — more clusters inevitably lower the individual variances — so the minimum cost is not the criterion.

What we look for is the point where the **decrease slows down markedly**: beyond it, additional clusters bring little improvement.



On these four blobs the curve bends at $K = 4$, where the cost is 564.91 — the number of groups the data was generated with.



The Silhouette Method Explained

The silhouette method measures how well the data is grouped, by computing a score **for each data point** saying how well it fits its own cluster compared with all the others. If K is right, each point should fit its cluster well and no other cluster well.

For a point i assigned to cluster C_I , the first term is the average distance to the other points of its own cluster:

$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d(i, j)$$

where $d(i, j)$ is the Euclidean distance. We expect it to be small if the cluster has low variance. The second term is the average distance to the *best alternative* cluster:

$$b(i) = \min_{J \neq I} \frac{1}{|C_J|} \sum_{j \in C_J} d(i, j)$$

The **silhouette score** compares the two:

$$s(i) = \begin{cases} \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} & \text{if } |C_I| > 1 \\ 0 & \text{otherwise} \end{cases} \quad -1 \leq s(i) \leq 1$$

- $s(i)$ close to **1**: $a(i) \ll b(i)$, the point fits its cluster far better than any other;
- $s(i)$ close to **-1**: $a(i) \gg b(i)$, another cluster would suit it better;
- $s(i)$ close to **0**: the point sits on the border between two natural clusters.

The score is set to 0 when $|C_I| = 1$ — a cluster containing only that point. That is an **assumption of neutrality**, not a measurement.

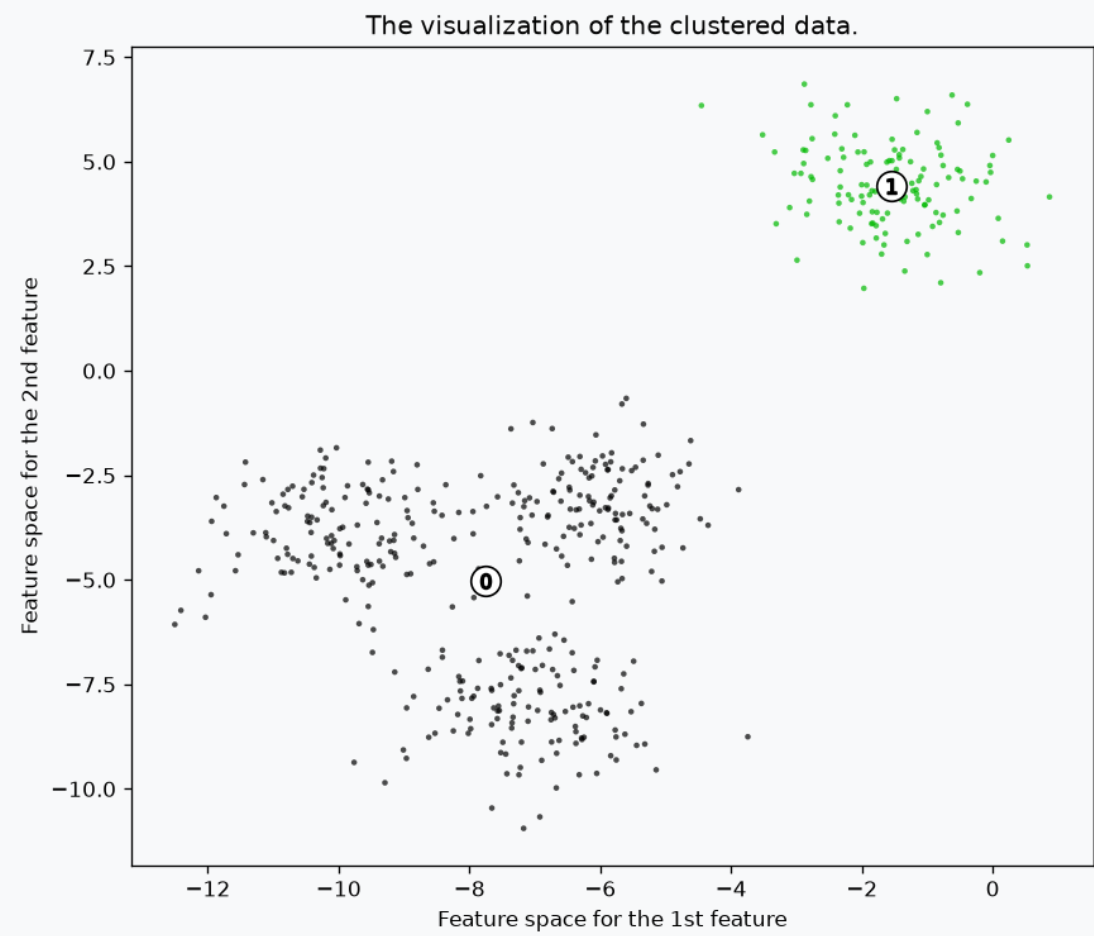
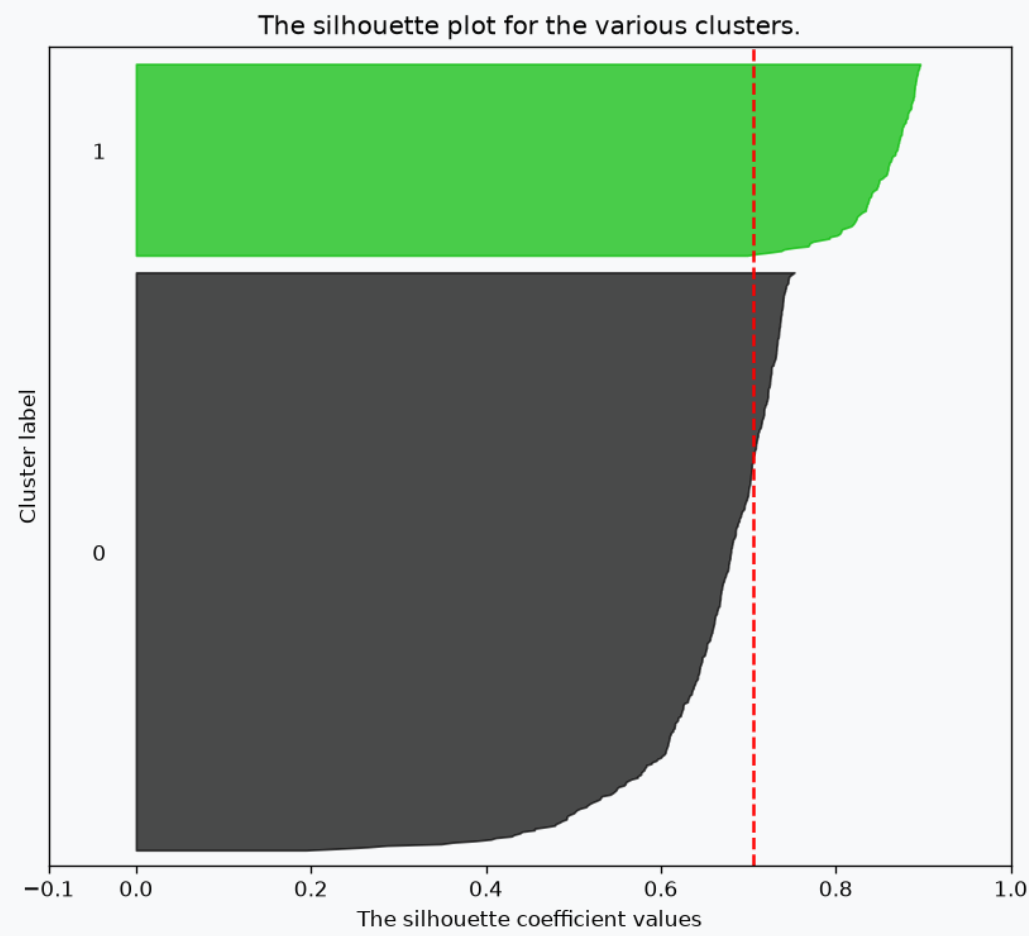


Silhouette Score in Action

The correct number of clusters is usually chosen by reading the **silhouette plot**: all the silhouette scores of the points of each cluster, sorted from largest to smallest, with the average as a dashed line.

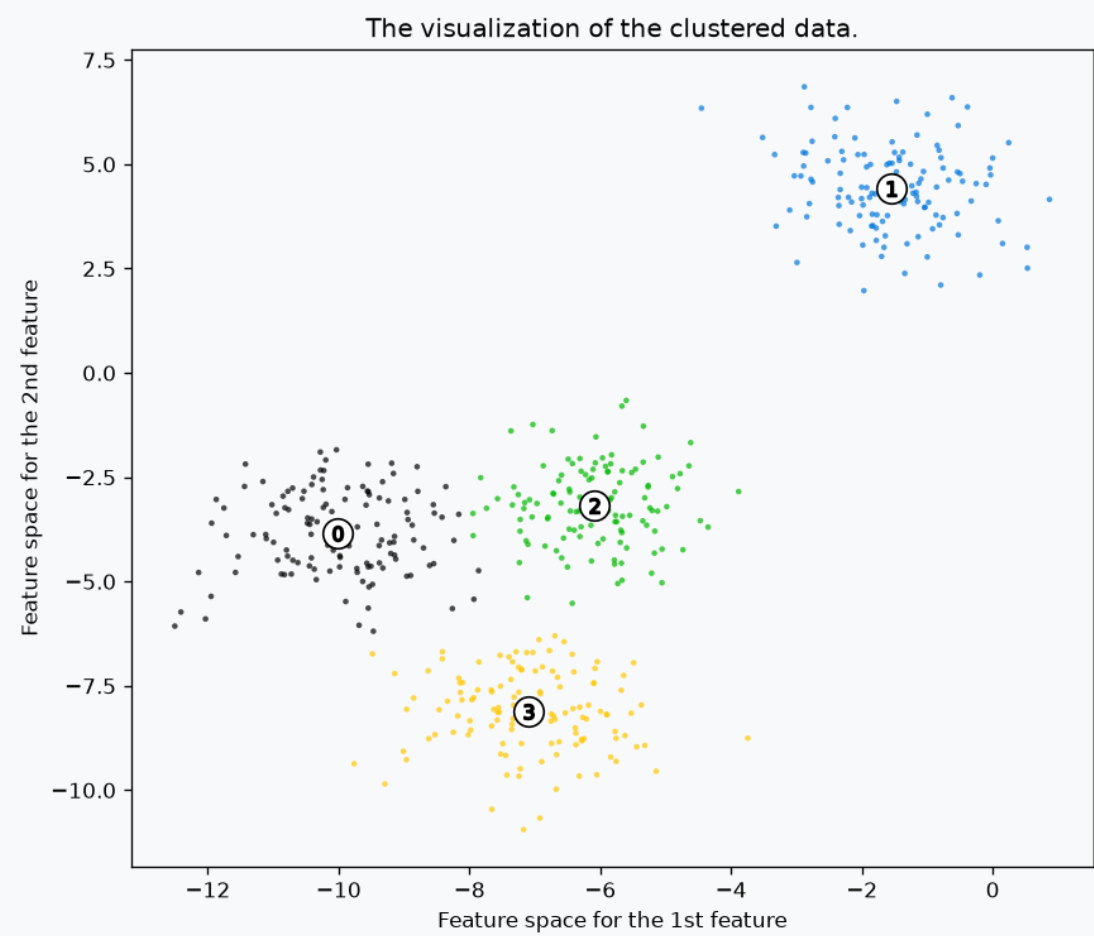
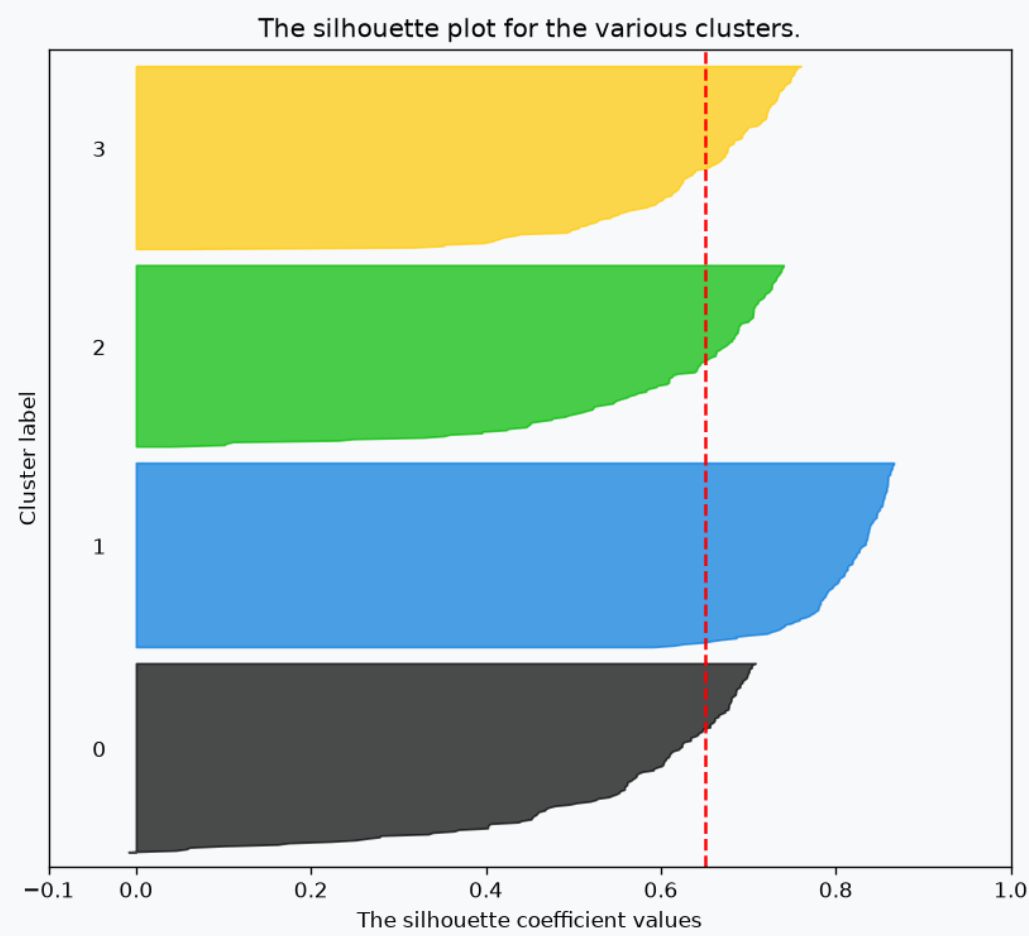
K = 2 — one huge cluster and one small one; the big cluster's scores sag well below the average.

Silhouette analysis for KMeans clustering on sample data with n_clusters = 2



K = 4 — every cluster is wide, every one crosses the average, and none is obviously split.

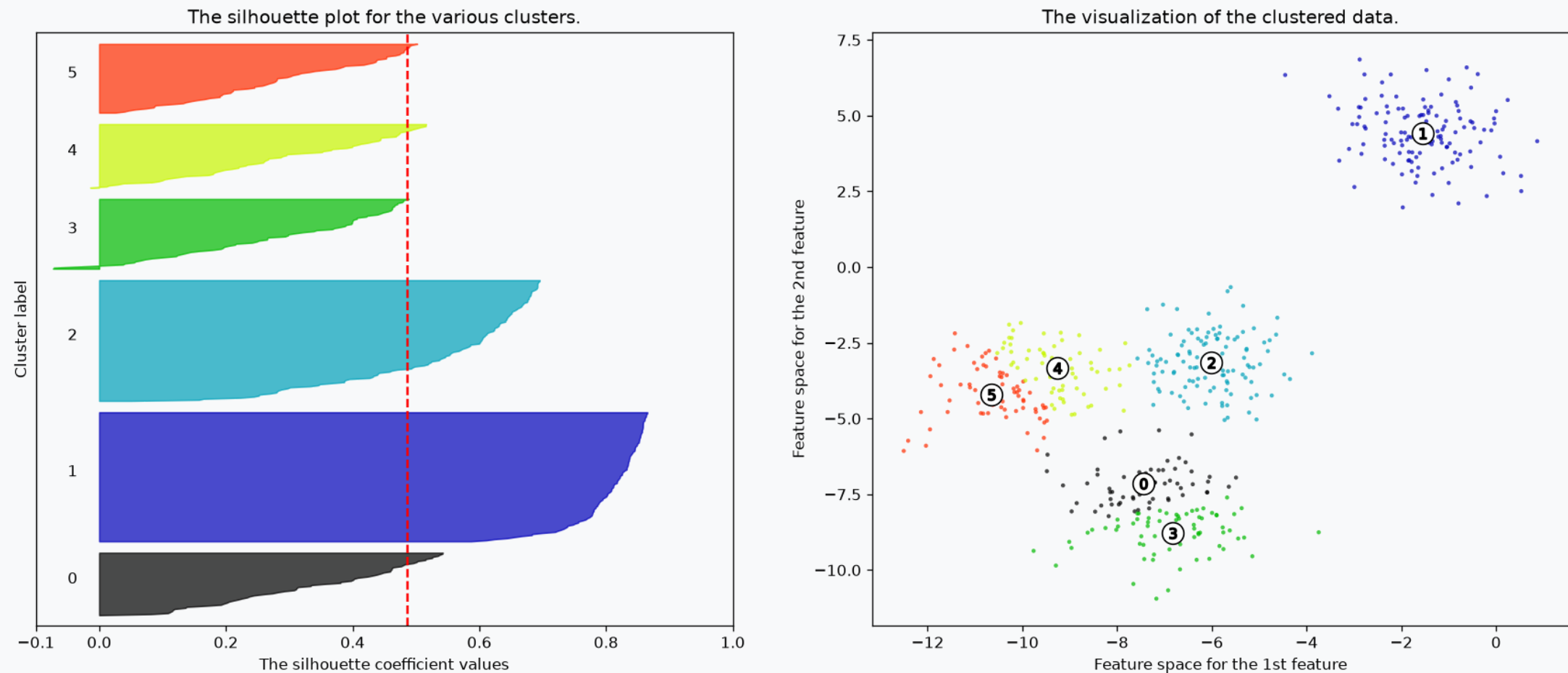
Silhouette analysis for KMeans clustering on sample data with n_clusters = 4



Silhouette Score in Action (cont.)

$K = 6$ — two clusters have fallen below the average: the data has been cut too finely.

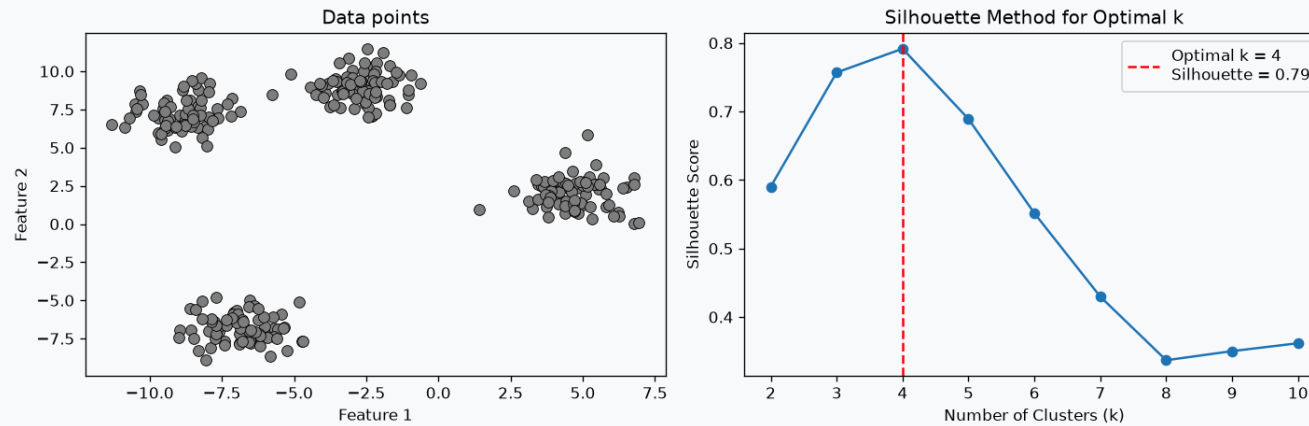
Silhouette analysis for KMeans clustering on sample data with $n_clusters = 6$



A cluster whose bar is thin, or whose scores fall mostly under the average line, is a sign that K is wrong. Note also the right-hand panels: at $K = 6$ the algorithm has split two of the four real blobs, which is visible in the scatter long before it is visible in the average score.



Silhouette Score in Action: Choosing K



Averaging the silhouette scores over all points gives one number per K , and the curve has a maximum where the elbow had its bend: $K = 4$, with an **average silhouette of 0.79**.

Here the two heuristics agree, which is the comfortable case. When they disagree, neither is authoritative: they are two different summaries of the same geometry, and the choice goes back to what the clusters have to be used for.



Other Clustering Algorithms

K-means is the most popular clustering algorithm, and one among many. Others take different views of what a cluster is, and address cases where K-means struggles:

Hierarchical clustering

Builds a tree of nested groupings instead of one partition, so the number of clusters is chosen by cutting the tree, after the fact.

DBSCAN

Density-based: clusters are dense regions separated by sparse ones. It finds clusters of arbitrary shape, and labels outliers as noise rather than forcing them into a group.

Spectral clustering

Clusters in the space of the eigenvectors of a similarity graph, which handles groups that are connected but not compact.

- In the next lecture we see **Gaussian Mixture Models**, a probabilistic model for density estimation that can also be read as a clustering algorithm — one where a point belongs to each cluster with a probability, rather than belonging to exactly one.



Lab: Customer Segmentation

Let us apply K-means to a business problem: segmenting **200 mall customers** by their annual income and spending score, so that a marketing team can target each group differently. A "high income, low spending" group needs a different message from a "low income, high spending" one.

01

Load and explore

Read the Mall Customers dataset and look at the relationships with a `pairplot`.

03

Find the best K

Use both the elbow and the silhouette method on the scaled data.

02

Prepare and scale

Select the features and apply `StandardScaler`, so that income and score weigh the same.

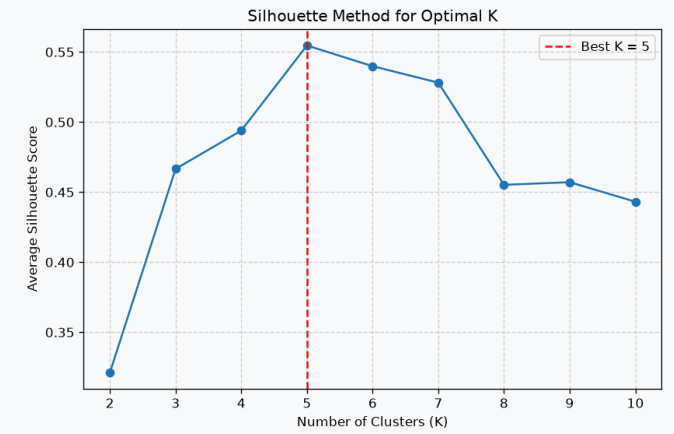
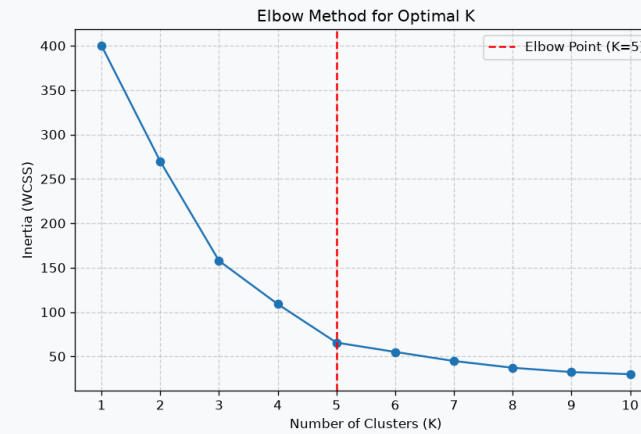
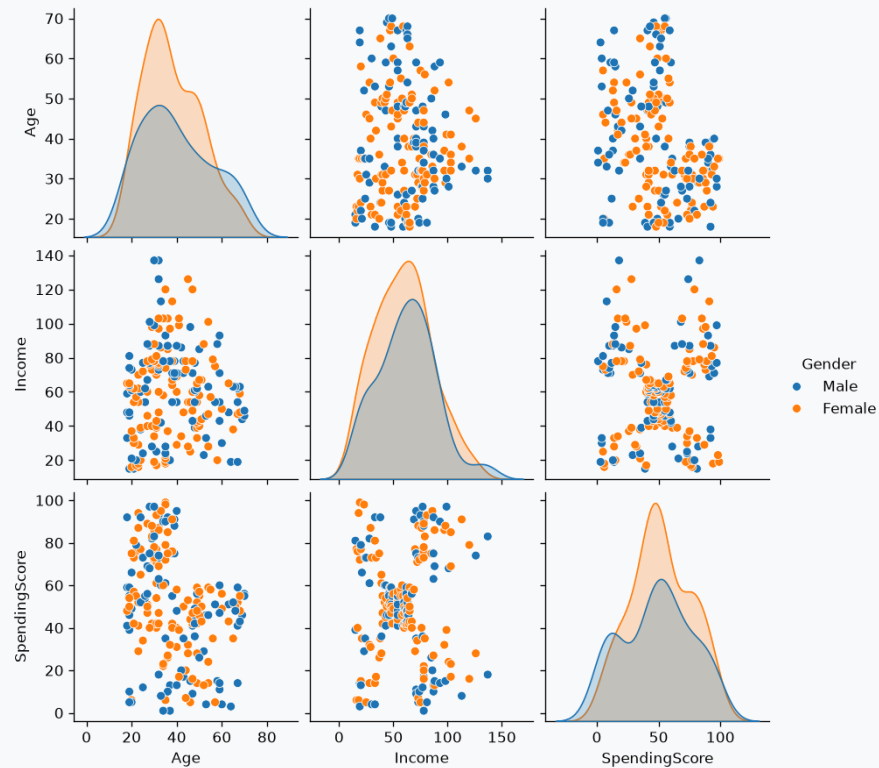
04

Fit and interpret

Train the final model and read the clusters as customer segments.



Lab: Exploring, Scaling, Choosing K



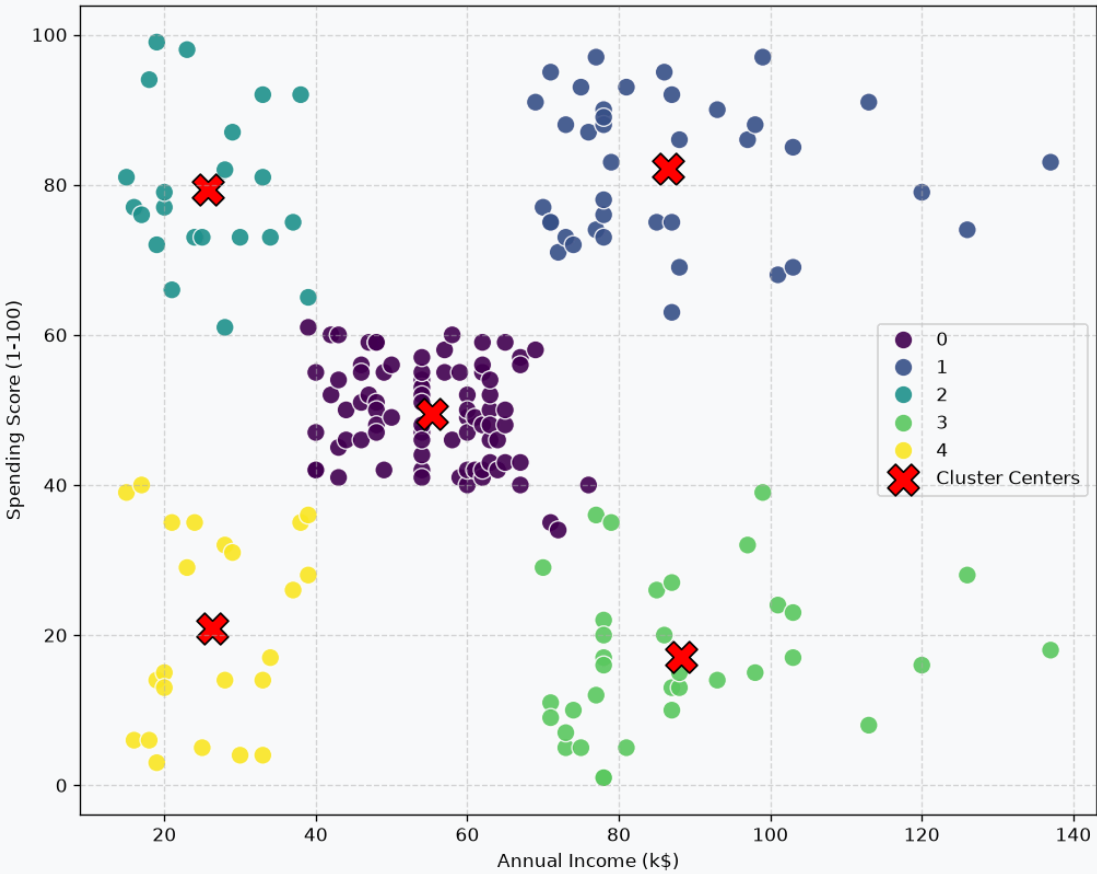
Scaling comes first: income runs in tens of thousands and the spending score from 1 to 100, so without `StandardScaler` the distance would be the income difference and nothing else.

The elbow bends at $K = 5$, and the average silhouette is highest at $K = 5$ as well. Both methods agree, so that is the model we fit.

The pairplot of age, income and spending score, split by gender. Income against spending score is where the structure is: the points fall into corners rather than one cloud.



Lab: The Five Segments

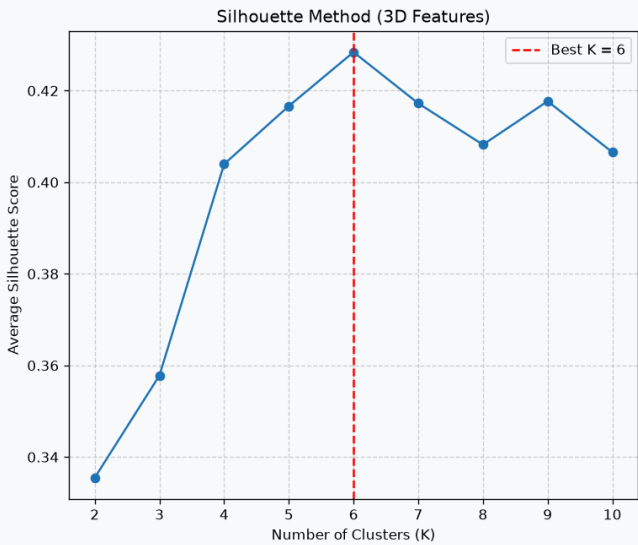
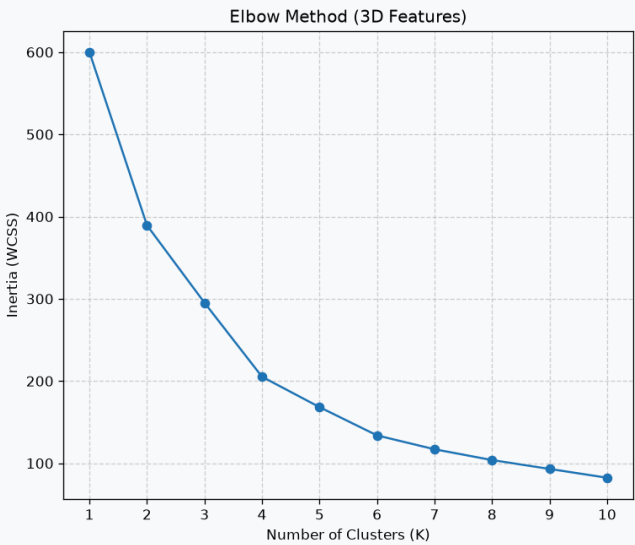


Cluster	Income	Spending	Reading
0	55.3	49.5	The Standard — the large, ordinary group
1	86.5	82.1	The Targets — money, and happy to spend it
2	25.7	79.4	The Risk — low income, high spending
3	88.2	17.1	The Careful Rich — money, and no spending
4	26.3	20.9	The Savers — cautious on both counts

The four corners plus the middle. K-means partitioned the customers into five groups a business can actually write different campaigns for — and the names are ours, not the algorithm's: it only returned five sets of points.



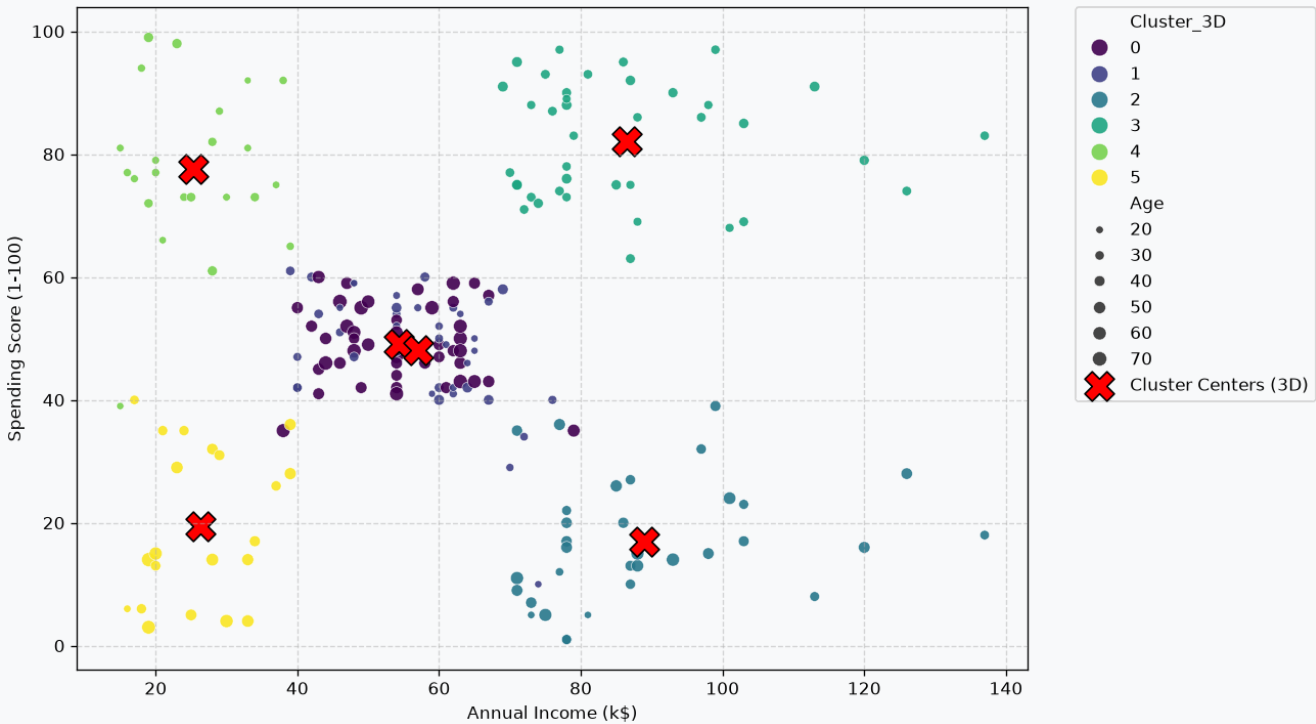
Lab: Going Further, in Three Dimensions



The 2-D analysis knew only about income and spending score. Adding **age** as a third feature — scaled with the other two — changes the answer: the optimal K moves from **5 to 6**.

This is the key finding of the 3-D analysis, and a general lesson: **changing the feature space changes the structure the data appears to have.**

Visual evidence



The same plot as before, with age drawn as the **size** of the dot. The single "Standard" blob in the middle has split into two: same average income and spending, different ages — younger customers in one group, older ones in the other.

Cluster	Age	Income	Spending
0	56.3	54.3	49.1
1	26.8	57.1	48.1

The two groups that the 2-D projection had hidden.



Conclusions and Next Steps



We Have Explored:

Clustering, and what it is for

K-means, its cost function and its inertia

The assignment and update steps, to convergence

The elbow method

The silhouette method

Segmenting customers, in two dimensions and in three

- In the next lecture we look at density estimation.

References

- Section 10.3.1 of [1] · Sections 2.3.9 and 9.2 of [2]
- Silhouette: [en.wikipedia.org/wiki/Silhouette_\(clustering\)](https://en.wikipedia.org/wiki/Silhouette_(clustering))

[1] James, G., Witten, D., Hastie, T., & Tibshirani, R. *An introduction to statistical learning*. Springer, 2013.

[2] Bishop, Christopher M. *Pattern recognition and machine learning*. Springer, 2006.

- Lecture notes: [Chapter 18 — Clustering](#)

